

# Physicalized Model Weights - cycles 2-4

2026-05-13

## Contents

|                                                |          |
|------------------------------------------------|----------|
| <b>Physicalized Model Weights - cycles 2-4</b> | <b>1</b> |
| Abstract . . . . .                             | 1        |
| Introduction . . . . .                         | 1        |
| Approach . . . . .                             | 2        |
| Findings . . . . .                             | 2        |
| Discussion . . . . .                           | 7        |
| Open Questions . . . . .                       | 7        |
| References . . . . .                           | 7        |
| Appendix: Implementation Details . . . . .     | 7        |

## Physicalized Model Weights - cycles 2-4

### Abstract

Cycles 2-4 converted the physicalized-weights research run from a broad design question into a narrow, validated architecture target. The work first defined “physicalizing model weights” as moving stable inference components from fully programmable software execution into fixed, semi-fixed, near-memory, analog, or hybrid hardware structures. It then built a normalized break-even model for when such physicalization can beat optimized programmable inference. That model preserved the strongest null hypothesis: software, runtime, compiler, scheduler, and programmable-accelerator improvements may capture most practical gains without fixed weights.

The next cycle used the model to rank physicalization targets. It rejected full frontier-model permanence, dynamic attention, high-churn adapters, vocabulary heads, and training state. The top ranked target was a fixed or semi-fixed safety/filter classifier submodel with programmable fallback.

The final cycle in this range specified and validated a hybrid safety-filter architecture. The design keeps the physicalized classifier behind a narrow memory-mapped boundary, with update, rollback, audit, fallback, and final routing under programmable control. The auditor validated M-ARCH-1 with no critical or moderate defects. Remaining gaps are expected: no HDL or latency prototype exists yet, numeric costs are normalized placeholders, and pytest is unavailable in the environment, so tests were run through stdlib harnesses.

### Introduction

The directive asks whether useful parts of neural-network inference can be “physicalized” into hardware rather than repeatedly executed as software kernels over CPU/GPU memory hierarchies. In this report, physicalization means encoding some stable part of an inference workload directly into hardware structure, storage, topology, data movement, or compute behavior. It does not mean assuming that an entire frontier language model should be per-

manently burned into a chip.

The cycle range reported here is labelled cycles 2-4 by the report input. Several source sessions describe their local milestone pass as “Cycle 1 / max 3”; that wording refers to each milestone audit pass, not to the global cycle range of this report.

The work followed three steps:

1. Build the foundation: taxonomy, null hypothesis, and break-even model.
2. Narrow the design space: ranked candidate and anti-target selection.
3. Specify an architecture: hybrid safety-filter classifier with programmable fallback.

Source sessions: researcher 5f531a46-18b8-42c7-979a-1133731a6b17, worker 38db3311-f9ad-4b5b-8f4d-72c504a16555, auditor e872025f-f4e4-4051-88d5-95fea376fe29; researcher 7622b5ff-2e8f-41cd-a531-49430774d52e, worker 56c69dfe-8d31-4693-9acb-9f668d873470, auditor fe2a9eb7-7922-41b9-8546-0a7255e6f0b6; researcher 12ce7e94-e5e9-4df7-907a-cec4865603e4, worker ce2446bb-153e-499a-92ac-13918e7225c0, auditor 70881275-75ff-4b15-aeaa-06f54dc39934.

## Approach

The work used the directive’s engineering constraint: compare physicalized designs against strong programmable baselines before crediting any hardware win. Those baselines included optimized software inference, AI-aware runtime and scheduling improvements, and programmable accelerators.

The primary analytical mechanism was:

$$N * (C_{\text{prog}} - C_{\text{phys}}) > C_{\text{fixed}} + C_{\text{update}} + C_{\text{yield}} + C_{\text{integration}}$$

Here,  $N$  is the number of requests served before a material weight update.  $C_{\text{prog}}$  is the best programmable per-request cost after software/runtime savings.  $C_{\text{phys}}$  is the physicalized per-request cost after conversion, utilization, yield/repair, and fallback penalties. The mechanism says fixed or semi-fixed hardware only wins when per-request savings amortize fixed substrate, update, yield, and integration costs.

The cycle also used regenerable artifacts instead of claims alone:

- Python models and CSV/JSON/PNG outputs for break-even and target scoring.
- A Wolfram script for symbolic break-even special cases.
- A Graphviz diagram for architecture structure.
- A deterministic fallback-policy simulator for safety/filter routing behavior.
- Test files executed through a stdlib harness because pytest is not installed.

## Findings

### Cycle 2: Taxonomy, Null Hypothesis, and Break-Even Model

Cycle 2 established the foundation for the rest of the run. The researcher session directed the worker to define physicalization levels, list inference components, state the strongest null hypothesis, and build a first executable break-even model. The worker produced physicalized-weights/docs/taxonomy\_and\_null.md, physicalized-weights/scripts/breakeven\_model.py, physicalized-weights/scripts/symbolic\_breakeven.wls, physicalized-weights/tests/test\_breakeven\_model.py, and generated CSV/JSON/PNG outputs.

The taxonomy separated fixed digital logic, ROM-coded weights, SRAM/eDRAM-resident weights, FPGA/eFPGA overlays, RISC-V/custom accelerators, chiplet or cartridge weights,

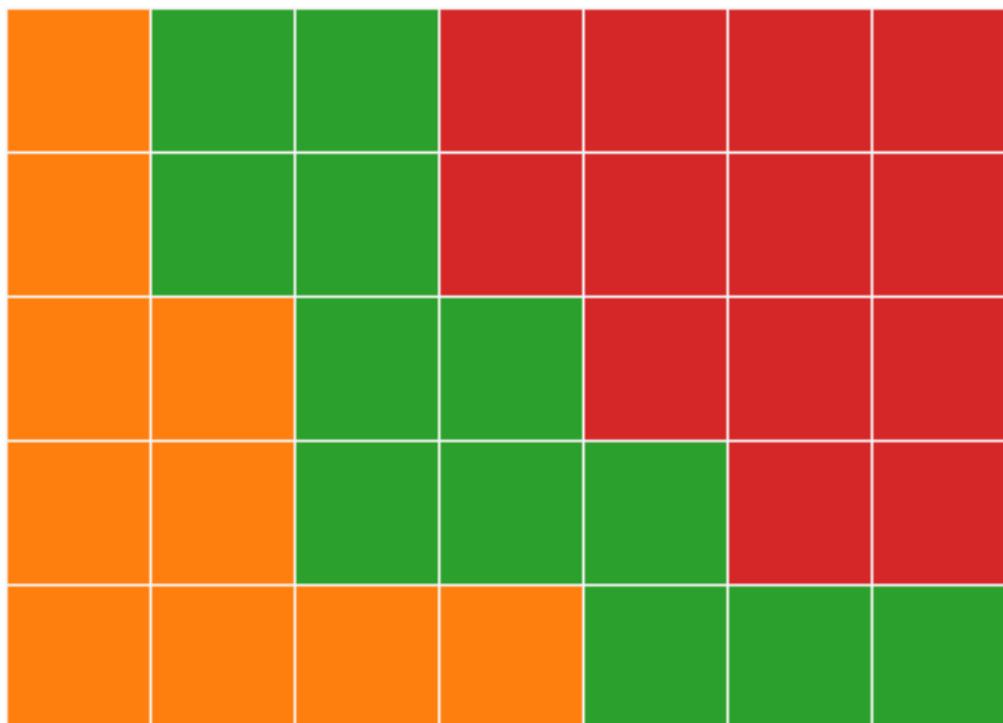
analog in-memory compute, photonic/mixed-signal systems, and hybrid fixed/programmed systems. It also decomposed inference into static base weights, adapters, mixture-of-experts blocks, embeddings, routers, speculative draft models, attention/KV paths, tokenizers, and safety/verifier models.

The null hypothesis was central: optimized programmable serving, AI-aware operating systems and runtimes, compiler placement, memory managers, batching, routing, quantization, caching, and programmable accelerators may eliminate most of the advantage that fixed weights appear to have.

The executable model compared six strategies:

- `programmable_unoptimized`
- `software_optimized`
- `programmable_accelerator`
- `fixed_digital_weights`
- `analog_in_memory`
- `hybrid_physicalized_submodel`

It emitted 3,360 data rows. For the checked winner counts, `fixed_digital_weights` won 208 rows, `programmable_accelerator` won 192 rows, and `software_optimized` won 160 rows. Analog in-memory and hybrid physicalized submodels did not win under the reported default/bad-penalty sweep. The worker interpreted this as a useful negative result: array-level analog savings are not enough when conversion, yield, repair, or fallback penalties dominate.



**Figure 1:** Break-even regions over update cadence and request volume showing which strategy minimizes normalized cost/energy at software memory savings = 0.35.

The Wolfram script derived the symbolic threshold:

```
N_break_even = fixedCost / (cProg - cPhys)
```

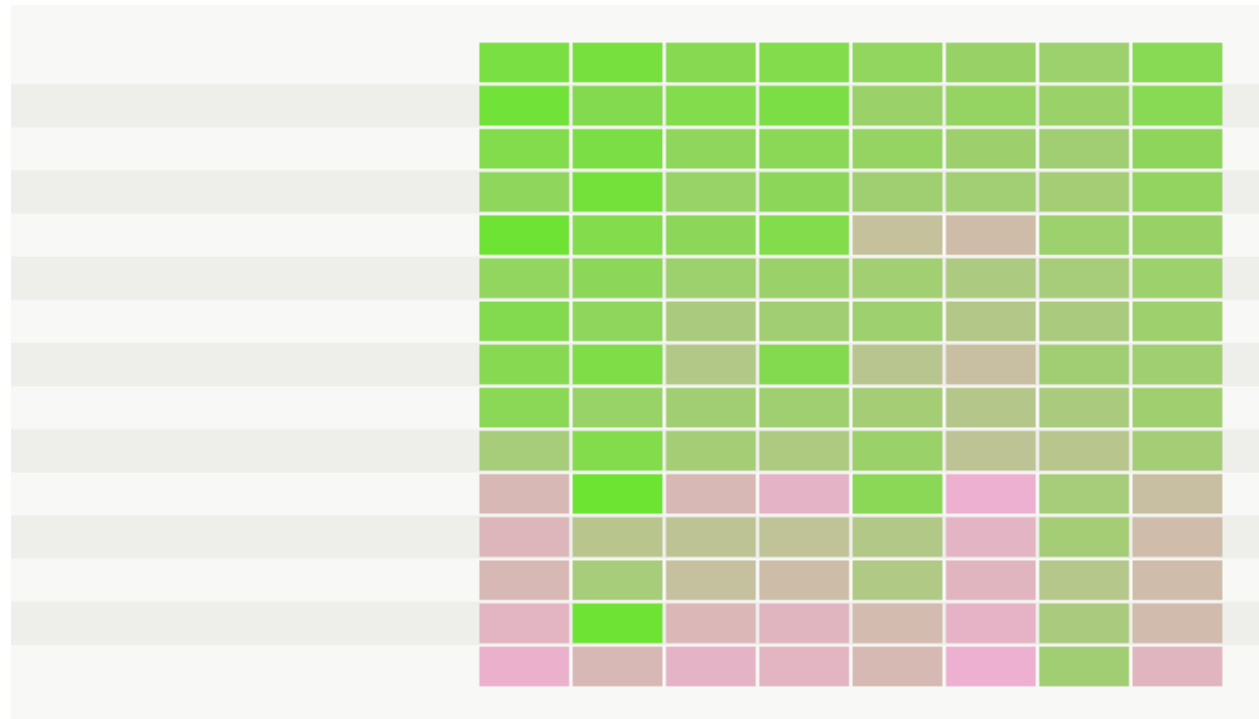
It checked the required special cases: zero request volume cannot amortize nonzero fixed cost; no finite positive break-even exists when the programmable cost is already less than or equal to the physicalized cost; shorter update intervals penalize fixed strategies; and 50% software memory-movement savings raises the threshold for physicalization.

The auditor validated M-TAX-1, M-MODEL-1, and M-BASE-1. No critical or moderate defects were found. Minor issues were environmental: pytest was unavailable, and the PNG fallback was dependency-light.

### Cycle 3: Target Ranking

Cycle 3 turned the break-even model into a ranked target list. The researcher directed the worker to score candidate components by update stability, reuse volume, approximation tolerance, integration complexity, energy upside versus baseline, resistance to software-baseline improvements, and evidence quality.

The worker produced `physicalized-weights/scripts/target_scoring.py`, `physicalized-weights/docs/target_ranking.md`, `physicalized-weights/tests/test_target_scoring.py`, and generated `target_scores.csv`, `target_scores_summary.json`, and `target_score_heatmap.png`.



**Figure 2:** Heatmap of scored candidate and anti-target components across physicalization viability axes, with total score sorted descending.

The top ranked candidates at 35% software/runtime memory-movement savings were:

| Rank | Component                                       | Score |
|------|-------------------------------------------------|-------|
| 1    | Fixed safety/filter classifier submodels        | 3.775 |
| 2    | Small always-on wake-word or router models      | 3.772 |
| 3    | Repeated retrieval/reranking feature transforms | 3.545 |
| 4    | Mixture-of-experts router or dispatch logic     | 3.354 |

The rejected anti-targets were:

|  | Rank | Anti-target                                                         | Score |
|--|------|---------------------------------------------------------------------|-------|
|  | 11   | Full frontier LLM dense weights burned permanently into fixed logic | 1.377 |
|  | 12   | Frequently updated tenant-specific fine-tunes or adapters           | 1.127 |
|  | 13   | High-churn vocabulary/logit-head variants                           | 1.126 |
|  | 14   | Dynamic attention over live context as fixed physical logic         | 1.045 |
|  | 15   | Training/optimizer state and gradient computation                   | 0.472 |

The decision from this cycle was not to pursue full-model physicalization. The recommended target became a fixed or semi-fixed safety/filter classifier submodel with programmable fallback. The rationale was that such a classifier can be small, isolated, repeatedly invoked, versioned conservatively, and routed around when confidence, health, policy version, or audit requirements fail.

The auditor validated M-TARGET-1. It confirmed 10 candidates, 5 anti-targets, regenerable CSV/JSON/PNG outputs, schema and ranking tests, and software-savings sensitivity. No critical or moderate defects were found. Minor issues were that pytest remained unavailable and the PNG heatmap was valid but visually minimal.

#### Cycle 4: Hybrid Safety-Filter Architecture

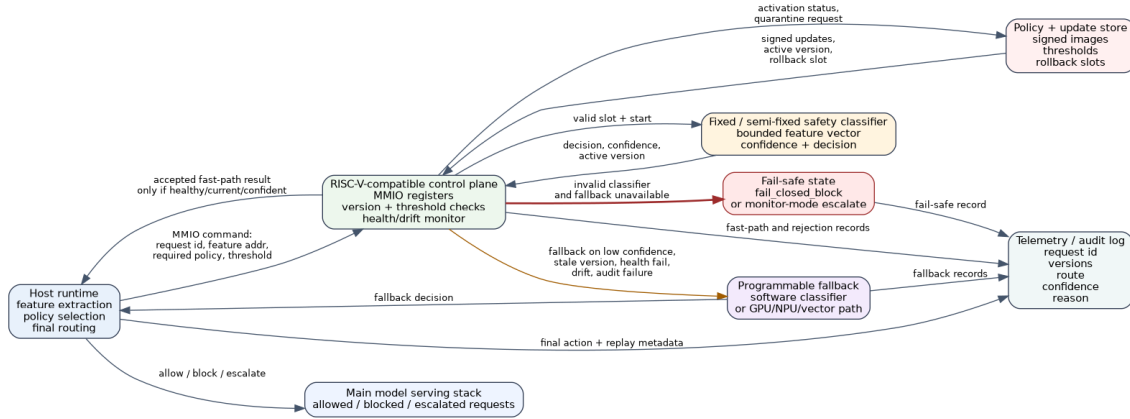
Cycle 4 used the top ranked target to define a concrete architecture. The researcher directed the worker to specify the system boundary, open control-plane choice, inputs, outputs, registers, update and rollback paths, fallback logic, invariants, and prototype roadmap.

The worker produced:

- physicalized-weights/docs/hybrid\_safety\_filter\_architecture.md
- physicalized-weights/docs/hybrid\_safety\_filter\_arch.dot
- physicalized-weights/docs/hybrid\_safety\_filter\_arch.png
- physicalized-weights/scripts/fallback\_policy\_sim.py
- physicalized-weights/tests/test\_fallback\_policy\_sim.py

- physicalized-weights/data/hybrid\_arch\_policy\_cases.csv
- physicalized-weights/data/hybrid\_arch\_summary.json

Hybrid safety-filter architecture with fixed/semi-fixed classifier fast path, programmable fallback, update/version control, and audit telemetry



**Figure 3:** Hybrid safety-filter architecture with fixed/semi-fixed classifier fast path, programmable fallback, update/version control, and audit telemetry.

The architecture places the physicalized classifier before the main model serving stack. The host runtime constructs bounded request features, invokes the classifier fast path, checks confidence/version/health/drift/audit state, and either accepts the physicalized decision or routes to software or programmable-accelerator fallback. The classifier does not emit user-facing content and does not own tokenization, prompt parsing, policy authoring, dynamic attention state, or final routing.

The control-plane decision was a host-controlled memory-mapped accelerator with a RISC-V-compatible path. RISC-V was chosen as an open integration anchor because it is a free and open instruction set architecture with public technical specifications [1], [2]. The report sources did not treat RISC-V as automatically optimal; they used it as a credible open management and accelerator-integration substrate.

The register map covered device identity, ABI version, status, control, request ID, feature address and length, policy slot, required and active policy versions, threshold, confidence, decision, fallback reason, audit ring state, and rollback slot.

The simulator exercised 11 policy cases. Its summary reported:

| Route                  | Count |
|------------------------|-------|
| physicalized_fast_path | 2     |
| programmable_fallback  | 7     |
| fail_safe              | 2     |

Accepted fast-path cases were limited to a healthy high-confidence classifier and a valid fast path when fallback was unavailable. Stale policy, low confidence, zero confidence, failed health, drift alarm, host-forced fallback, and audit logging failure all routed away from the physicalized output. Invalid classifier output with unavailable fallback entered fail-safe.

The auditor validated M-ARCH-1. It reproduced Graphviz diagram generation, confirmed the PNG dimensions as 1737 by 623 RGBA, reran the simulator, confirmed 11 policy cases, and ran all six test functions through a stdlib harness. No critical or moderate defects were found.

## Discussion

Cycles 2-4 narrowed the research thesis substantially. The work no longer asks whether arbitrary model weights should be fixed in hardware. It asks whether a small, stable, high-volume, auditable classifier-like submodel can justify a physicalized fast path while preserving programmable update, fallback, audit, and routing control.

The central result is conditional. Fixed digital physicalization appears only in regions where request volume and update interval are large enough to amortize fixed cost. Software-optimized and programmable-accelerator baselines remain strong. Analog in-memory did not win under the reported default penalty sweep, which means later analog work must account for ADC/DAC conversion, yield/repair, drift, and fallback overhead before claiming advantage [6].

The validated architecture also shifts the likely product shape. The credible system is not a standalone “weight chip.” It is closer to an AI-aware runtime or host control plane that can route work across ordinary programmable execution, programmable accelerators, and optional fixed/semi-fixed modules. Physicalization must therefore buy something beyond what that control plane already provides. The current candidate buys only a narrow possibility: lower repeated classifier cost under high reuse, slow update cadence, bounded feature extraction, low fallback rate, and auditable failure handling.

## Open Questions

The main open question is whether the safety/filter classifier fast path remains beneficial after prototype-level overheads are measured. The next milestone, M-PROTO-1, should quantify feature extraction cost, audit logging cost, fallback frequency, fast-path utilization, and control complexity.

The current model uses normalized cost/energy units. It does not yet use sourced joule, area, latency, packaging, or process-specific estimates. That is acceptable for boundary finding, but not enough for device-level claims.

No HDL, Verilator, Yosys, or synthesis evidence has been produced in this cycle range. The audit guidance says HDL should remain small if introduced next: a register-map or finite-state-machine sketch checked with Verilator/Yosys/Graphviz would be enough.

pytest is unavailable in the current environment. Workers and auditors used stdlib harnesses instead. This is an environment gap, not a reported correctness failure.

## References

[1] RISC-V International, “RISC-V FAQ,” RISC-V International. <https://riscv.org/about/faq/>

[2] RISC-V International, “RISC-V Technical Specifications,” RISC-V International. <https://docs.riscv.org/refe>

[6] IBM Research, “How can analog in-memory computing power transformer models?,” IBM Research. <https://research.ibm.com/blog/how-can-analog-in-memory-computing-power-transformer-models>

## Appendix: Implementation Details

### Code Organization

Authored physicalized-weights scripts:

| File                                                | Lines | Purpose                                                                 |
|-----------------------------------------------------|-------|-------------------------------------------------------------------------|
| physicalized-weights/scripts/breakeven_model.py     | 384   | Normalized break-even sweep across six inference strategies.            |
| physicalized-weights/scripts/target_scoring.py      | 283   | Candidate and anti-target scoring model using break-even calibration.   |
| physicalized-weights/scripts/fallback_policy_sim.py | 342   | Deterministic fallback/fail-safe simulator for the hybrid architecture. |
| physicalized-weights/scripts/symbolic_breakeven.wls | 28    | Wolfram symbolic break-even derivation and special-case checks.         |

#### Authored tests:

| File                                                   | Lines | Purpose                                                   |
|--------------------------------------------------------|-------|-----------------------------------------------------------|
| physicalized-weights/tests/test_breakeven_model.py     | 108   | Break-even invariant and schema tests.                    |
| physicalized-weights/tests/test_target_scoring.py      | 133   | Ranking, anti-target, software-savings, and schema tests. |
| physicalized-weights/tests/test_fallback_policy_sim.py | 117   | Routing invariant and schema tests for fallback policy.   |

#### Authored docs and diagram source:

| File                                                           | Lines | Purpose                                                                                |
|----------------------------------------------------------------|-------|----------------------------------------------------------------------------------------|
| physicalized-weights/docs/taxonomy_and_null.md                 | 95    | Taxonomy, null hypothesis, component mapping, baselines, and falsification criteria.   |
| physicalized-weights/docs/target_ranking.md                    | 61    | Ranked candidates and anti-targets with top architecture target.                       |
| physicalized-weights/docs/hybrid_safety_filter_architecture.md | 149   | Architecture proposal with interfaces, invariants, update/fallback paths, and roadmap. |
| physicalized-weights/docs/hybrid_safety_filter_arch.dot        | 127   | Graphviz source for the hybrid safety-filter diagram.                                  |

## Generated Data and Figures



| Artifact                                                | Reported content             |
|---------------------------------------------------------|------------------------------|
| physicalized-weights/data/breakeven_grid.csv            | 3,360 data rows plus header. |
| physicalized-weights/data/target_scores.csv             | 15 target rows plus header.  |
| physicalized-weights/data/hybrid_arch_policy_cases.csv  | 11 policy rows plus header.  |
| physicalized-weights/data/breakeven_update_volume.png   | 402 x 294 RGB PNG.           |
| physicalized-weights/data/target_score_heatmap.png      | 696 x 400 RGB PNG.           |
| physicalized-weights/docs/hybrid_safety_filter_arch.png | 1737 x 623 RGBA PNG.         |

## Test Results

Cycle 2 worker and auditor reported:

- python3 physicalized-weights/scripts/breakeven\_model.py passed and emitted 3,360 rows.
- wolfram-batch -script physicalized-weights/scripts/symbolic\_breakeven.wls passed.
- Stdlib test runner passed all 5 break-even tests.
- pytest failed because it is not installed.

Cycle 3 worker and auditor reported:

- python3 physicalized-weights/scripts/target\_scoring.py passed and emitted 15 rows.
- Stdlib harness passed all 5 target-scoring tests.
- pytest failed because it is not installed.

Cycle 4 worker and auditor reported:

- dot -Tpng physicalized-weights/docs/hybrid\_safety\_filter\_arch.dot -o physicalized-weights/docs/hybrid\_safety\_filter\_arch.png passed.
- python3 physicalized-weights/scripts/fallback\_policy\_sim.py passed and emitted 11 policy cases.
- Stdlib harness passed all 6 fallback-policy tests.
- pytest failed because it is not installed.

## Session References

| Cycle | Role       | Session ID                           | Contribution                                                                  |
|-------|------------|--------------------------------------|-------------------------------------------------------------------------------|
| 2     | Researcher | 5f531a46-18b8-42c7-979a-1133731a6b17 | Defined foundational taxonomy, null hypothesis, and break-even model brief.   |
| 2     | Worker     | 38db3311-f9ad-4b5b-8f4d-72c504a16555 | Built taxonomy doc, break-even model, Wolfram derivation, tests, and outputs. |
| 2     | Auditor    | e872025f-f4e4-4051-88d5-95fea376fe29 | Validated taxonomy/model/baseline milestones with minor issues only.          |

| Cycle | Role       | Session ID                           | Contribution                                                                       |
|-------|------------|--------------------------------------|------------------------------------------------------------------------------------|
| 3     | Researcher | 7622b5ff-2e8f-41cd-a531-49430774d52e | Directed target and anti-target ranking from break-even variables.                 |
| 3     | Worker     | 56c69dfe-8d31-4693-9acb-9f668d873470 | Built target scoring, ranking doc, tests, and heatmap outputs.                     |
| 3     | Auditor    | fe2a9eb7-7922-41b9-8546-0a7255e6f0b6 | Validated M-TARGET-1 and recommended architecture work.                            |
| 4     | Researcher | 12ce7e94-e5e9-4df7-907a-cec4865603e4 | Directed hybrid safety-filter architecture around the top target.                  |
| 4     | Worker     | ce2446bb-153e-499a-92ac-13918e7225c0 | Built architecture note, Graphviz diagram, fallback simulator, tests, and outputs. |
| 4     | Auditor    | 70881275-75ff-4b15-aeaa-06f54dc39934 | Validated M-ARCH-1 and recommended moving to M-PROTO-1.                            |

### Cross-Reference Map

| Origin                                                                                         | Consuming artifact                                                                             | Flow                                                                           |
|------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| physicalized_model_weights_long_exposure_prompt.md                                             | plan_of_record.md                                                                              | Directive converted into milestones and constraints.                           |
| plan_of_record.md                                                                              | promise_ledger.jsonl                                                                           | Milestones tracked through validation events.                                  |
| physicalized-weights/docs/taxonomy_and_null.md                                                 | physicalized-weights/scripts/breakeven_model.py                                                | Taxonomy and null hypothesis define strategy classes and falsification checks. |
| physicalized-weights/scripts/breakeven_model.py                                                | physicalized-weights/data/breakeven_grid.csv, physicalized-weights/data/breakeven_summary.json | Executable sweep emits calibration data.                                       |
| physicalized-weights/data/breakeven_grid.csv, physicalized-weights/data/breakeven_summary.json | physicalized-weights/scripts/target_scoring.py                                                 | Break-even outputs calibrate candidate scores.                                 |
| physicalized-weights/scripts/target_scoring.py                                                 | physicalized-weights/docs/target_ranking.md                                                    | Target scoring selects the fixed/semi-fixed safety-filter classifier.          |

| Origin                                                                 | Consuming artifact                                                     | Flow                                                          |
|------------------------------------------------------------------------|------------------------------------------------------------------------|---------------------------------------------------------------|
| physicalized-<br>weights/docs/target_ranking.md                        | physicalized-<br>weights/docs/hybrid_safety_<br>filter_architecture.md | Top target becomes the<br>architecture focus.                 |
| physicalized-<br>weights/docs/hybrid_safety_<br>filter_architecture.md | physicalized-<br>weights/scripts/fallback_<br>policy_sim.py            | Architecture invariants<br>become executable policy<br>cases. |
| physicalized-<br>weights/docs/hybrid_safety_<br>filter_arch.dot        | physicalized-<br>weights/docs/hybrid_safety_<br>filter_arch.png        | Graphviz source regenerates<br>the architecture figure.       |